

DAY 15 — ASYNCHRONOUS AI JOB PROCESSING

1. PROBLEM WE ARE SOLVING

Earlier (Day 14), your flow was:

Client → Controller → AI Service → External API → Response

✗ Problems with this approach:

- Blocking request thread
- Slow response time (AI latency)
- Poor scalability
- No retry mechanism
- Failure = immediate failure to user

DAY 15 GOAL

Convert system into:

Client → API → Queue → Worker → AI → Store → Client polls

CORE IDEA

👉 Decouple request from processing

Instead of doing work immediately:

- Accept request 
- Queue job 
- Process in background 
- Provide status via polling 

2. CORE BUILDING BLOCKS

◇ 1. AIJob (Domain Entity)

📁 Core/Entities/AIJob.cs

Purpose:

Represents a **unit of work**

```
public class AIJob
{
    public Guid Id { get; set; }
    public string Prompt { get; set; }
    public string Status { get; set; }
    public string? Result { get; set; }
    public string? Error { get; set; }

    public int RetryCount { get; set; }
    public int MaxRetries { get; set; }
}
```

🔗 Why this exists:

Instead of processing directly:

👉 We **track state over time**

Pending → Processing → Completed / Failed

◇ 2. Queue (IAIQueue + AIQueue)

📁 Interface:

```
public interface IAIQueue
{
    void Enqueue(AIJob job);
    AIJob? Dequeue();
}
```

📁 Implementation:

ConcurrentQueue<AIJob>

🔗 Why queue?

Decouples:

API layer **×** dependent on AI execution

into:

API → Queue → Worker

◇ 3. Job Store (IAIJobStore + AIJobStore)

📁 Uses:

ConcurrentDictionary<Guid, AIJob>

🧠 Why Job Store?

Because:

👉 API needs to **check status later**

GET /status/{jobId}

◇ 4. Background Worker (AIWorker)

📁 Infrastructure/BackgroundServices/AIWorker.cs

💧 This is the HEART of Day 15

```
while (!stoppingToken.IsCancellationRequested)
{
    var job = _queue.Dequeue();
```

🧠 What it does:

1. Continuously runs
2. Picks jobs from queue
3. Processes them
4. Updates status

🔄 WORKER FLOW

Dequeue job

→ Mark Processing

→ Call AI Service

→ Save result

→ Mark Completed

3. RETRY + FAILURE HANDLING (ADVANCED PART)

Retry Logic

```
job.RetryCount++;
```

Condition:

```
if (job.RetryCount < job.MaxRetries)
```

👉 Requeue job:

```
_queue.Enqueue(job);
```

✖ Final Failure

```
job.Status = "Failed";  
job.Error = "Job failed after X attempts";
```

WHY RETRY?

External APIs (AI, network) are:

- Unreliable
- Temporarily failing
- Rate limited

Without retry:

1 failure → permanent failure ❌

With retry:

Fail → Retry → Retry → Success/Fail ✅

Dead Letter Queue (DLQ concept)

You implemented:

After max retries → mark as Failed

👉 This is equivalent to:

DLQ in Kafka / RabbitMQ

◇ 4. AI SERVICE LAYER

📁 AiService.cs

💧 Responsibilities:

1. Caching

```
var cachedResult = await _cache.GetAsync(cacheKey);
```

2. Provider call

```
var aiResult = await _provider.GenerateAsync(...)
```

3. Safe caching

```
if (!aiResult.StartsWith("⚠"))
```

🧠 Why this design?

Separates:

Concern	Layer
API call	Provider
Business logic	Service
Caching	Service

◇ 5. PROVIDER LAYER

📁 OpenAIProvider.cs

Responsibility:

```
_httpClient.PostAsJsonAsync(...)
```

Why separate provider?

Allows:

- Swap OpenAI → Azure → Local LLM
- Test easily
- Mock failures (you did this ✓)

◇ 6. CONTROLLER FLOW

 AIController.cs

Generate API

```
var job = new AIJob { ... };
```

```
_jobStore.Add(job);  
_queue.Enqueue(job);
```

Important:

 Controller does NOT call AI anymore

Status API

```
var job = _jobStore.Get(jobId);
```

Response:

```
{  
  "status": "Completed",  
  "result": "...",  
  "error": null  
}
```



7. COMPLETE FLOW (END-TO-END)



Step-by-step:

1. Client sends request

POST /api/AI/generate

2. API creates job

Status = Pending

3. Job added to queue

Queue ← job

4. Worker picks job

Pending → Processing

5. AI Service executes

Cache → Provider → Response

6. Success case

Processing → Completed

7. Failure case

Processing → Retry → Retry → Failed

8. Client polls

GET /api/AI/status/{id}



8. ARCHITECTURE SUMMARY

Controller



Queue (In-Memory)



Worker (BackgroundService)



AI Service



Provider (HTTP)



External AI



9. WHAT MAKES THIS “PRODUCTION-LEVEL”



Non-blocking architecture

User doesn't wait



Retry mechanism

Handles transient failures



Fault isolation

Worker failures $\neq$ API failures



Observability

Logs + correlation ID



Extensibility

Easy to plug Redis / Kafka later